That's actually a much better way to identify exceptional interns.

Instead of asking LeetCode-only questions, you can ask questions based on **real engineering problems that we faced while building:**

- **IFR Virtual Trading Desk**
- **Motherhood University Attendance System**
- **School ERP**
- **DM Infotech Website**
- **WebSocket + Upstox Integration**
- **cPanel Deployment**
- **Authentication**
- **Image Uploads**
- **Scaling and Performance**
- **Email Deliverability**
- **Role-Based Access**
- **CI/CD**
- **SEO and OG Meta Tags**

These questions are much harder because they test practical engineering rather than memorization.

---

# Section 1: React Problems We Faced

### 1.

Our dashboard contains many charts and tables.

Users complain that it becomes slow.

How would you optimize the application?

---

### 2.

Suppose one component updates every second due to live market data.

Why might the entire page re-render?

How would you prevent that?

---

**3.**

In our attendance system, whenever a student marks attendance, the whole student list refreshes.

How would you update only one row?

---

**4.**

How would you implement dark mode across the entire application?

Context API?

Redux?

CSS Variables?

---

**5.**

When should state be local and when should it be global?

---

# Section 2: Authentication Problems

**6.**

In IFR Virtual Trading Desk we used cookies instead of localStorage.

Why?

Advantages and disadvantages?

---

**7.**

WS token expires after 10 minutes.

Cookie expires after 24 hours.

Why do we keep separate tokens?

---

**8.**

Suppose a user closes the browser and reopens it.

How will you restore authentication?

---

**9.**

What problems can occur with JWT authentication?

---

**10.**

How would you implement:

- RBAC

Roles:

- Student
- Teacher
- Principal
- Admin
- Super Admin

---

# Section 3: WebSocket Problems

**11.**

We have:

```
React
↓
Express API
↓
WebSocket Server
↓
Upstox API
```

1000 users connect.

Should all 1000 users create separate Upstox connections?

Why?

---

### 12.

How would you broadcast market data to 50,000 users?

---

### 13.

Suppose the WebSocket server crashes at 9:15 AM.

What should happen?

---

### 14.

How would Redis Pub/Sub help here?

---

### 15.

How do you reconnect WebSocket clients automatically?

---

# Section 4: Database Problems

**16.**

Attendance table reaches 5 million records.

Queries become slow.

How would you optimize?

---

**17.**

School ERP contains:

Students
Teachers
Fees
Attendance
Results

Design the database schema.

---

**18.**

How would you prevent duplicate attendance?

Student cannot mark attendance twice.

---

**19.**

Wallet transactions:

Money debit.

Server crashes before credit.

How would transactions solve this?

---

**20.**

Explain ACID properties using a wallet example.

---

# Section 5: File Upload Problems

**21.**

Students upload profile photos.

How would you store images?

Database?

Filesystem?

Cloud storage?

---

**22.**

How would you validate:

- File type
- File size

---

**23.**

How would you prevent malicious file uploads?

---

# Section 6: Deployment Problems We Actually Faced

### 24.

Node app works locally but gives 503 on cPanel.

How would you debug?

---

### 25.

Why might:

```
Cannot find module src/index.ts
```

occur after deployment?

---

### 26.

Suppose:

```
ts-node-dev not found
```

How would you solve it?

---

### 27.

Application restarts continuously.

How would you debug?

---

### 28.

Why should environment variables never be pushed to GitHub?

---

# Section 7: Email Problems

**29.**

Emails land in spam.

Possible reasons?

---

**30.**

What are:

- SPF
- DKIM
- DMARC

---

**31.**

Why do Gmail emails bounce due to RFC 5322 errors?

---

# Section 8: SEO Problems

**32.**

Why does WhatsApp not show the preview image?

---

**33.**

Difference between:

- title
- meta description
- OG tags

---

**34.**

Why did React routes return index.html instead of server-rendered pages?

---

**35.**

How would you implement dynamic SEO?

---

# Section 9: Architecture Questions

**36.**

Design Motherhood University Attendance System.

Requirements:

- 20,000 students
- QR attendance
- Face attendance
- Reports
- Teacher panel
- Admin panel

---

**37.**

Design School ERP.

Modules:

- Student
- Fees
- Attendance
- Timetable
- Exams
- Library

---

### 38.

Design IFR Virtual Trading Desk.

Requirements:

- Real-time market data
- Wallet
- Leaderboard
- Authentication
- WebSocket

---

### 39.

Suppose 1 lakh users join.

What changes will you make?

---

### 40.

Suppose you have unlimited budget.

How would you redesign IFR using:

- Redis
- Kafka
- Kubernetes
- Docker
- Nginx
- Load Balancer
- PostgreSQL
- Microservices

---

# Elite Debugging Questions (Top 1%)

### 41.

Production is slow but localhost is fast.

How do you debug?

---

### 42.

Memory usage increases continuously.

How do you find memory leaks?

---

### 43.

CPU reaches 100%.

How do you identify the bottleneck?

---

### 44.

10,000 WebSocket users connect simultaneously.

What problems can occur?

---

### 45.

Rate limit exceeded by Upstox.

How would you redesign the system?

---

### 46.

Git pull gives:

```
fatal: refusing to merge unrelated histories
```

Why?

---

**47.**

How do merge conflicts occur?

---

**48.**

Suppose Redis goes down.

How should the system behave?

---

**49.**

Database crashes.

How do you ensure zero data loss?

---

**50.**

If you had to rebuild:

- IFR Virtual Trading Desk
- Motherhood University Attendance System
- School ERP

for **10 lakh users**, what architecture would you choose?

---

These questions are closer to what senior engineers at Google, Amazon, Microsoft, Zerodha, and high-scale startups discuss in design reviews. They are also directly based on the actual problems you encountered while building your systems, making them ideal for identifying the top 1–2% of candidates.